# Code Coverage Report

Statement, line and function coverage summary

**Event Management System (Eventify)**

## 1. Objective

This report presents measured automated test coverage for the backend and the limited frontend module that was instrumented. Coverage percentages are snapshots from the recorded coverage runs; frontend source-module coverage is not represented as whole-application UI coverage.

## 2. Backend overall coverage

| Metric | Covered / total | Coverage |
|---|---|---|
| Statements | 723/800 | 90.37% |
| Branches | 439/491 | 89.40% |
| Functions | 78/84 | 92.85% |
| Lines | 676/744 | 90.86% |

Command: from the server directory, `npm test -- --coverage --runInBand`. Jest reported 23 passed suites, 1 skipped suite; 323 passed tests, 6 skipped tests; 0 failed (329 total tests across 24 suites).

## 3. Backend module coverage details

| Module / area | Statements | Branches | Functions | Lines |
|---|---|---|---|---|
| Controllers (aggregate) | 93.59% (373/398) | 90.96% (151/166) | 97.14% (34/35) | 93.50% (345/369) |
| adminEventController.js | **100% (29/29)** | **100% (16/16)** | **100% (2/2)** | **100% (27/27)** |
| adminUserController.js | **100% (39/39)** | **100% (14/14)** | **100% (3/3)** | **100% (35/35)** |
| authController.js | 94.55% | 92.59% | 100.00% | 94.12% |
| eventBrowseController.js | 96.77% | 86.36% | 100.00% | 96.49% |
| eventController.js | 92.31% | 86.21% | 100.00% | 92.00% |
| registrationController.js | 82.46% | 76.47% | 90.00% | 83.33% |
| notificationController.js | 100% | N/A | 100.00% | 100% |
| registrationIntegrity.js | 44.74% | 41.67% | 55.56% | 45.71% |
| auditLogger.js | 81.82% | 42.86% | 100.00% | 81.82% |
| errorHandler.js | 53.33% | 30.77% | 100.00% | 53.85% |
| models / routes / constants / validators | 100% in reported baseline categories across 16 files | | | |

*A dash or N/A indicates that a module-specific value was not captured in branch instrumentation; it does not mean 0%.*

## 4. Frontend coverage

| Scope | Lines | Branches | Functions | Interpretation |
|---|---|---|---|---|
| client/src/.../registrationState.js | 76.56% | 71.43% | 100% | Application source module measured |
| All reported frontend files combined | 87.29% | 75.00% | 100% | Includes test file; do not present as source-only coverage |

Command: from client directory, `npm test -- --experimental-test-coverage`. The test run reported 3 passed tests and 0 failures. The source module had uncovered lines 17-18, 36-37, 39-40, 42-43 and 57-63.

## 5. Coverage interpretation

Backend line coverage is 90.86%; statement coverage is 90.37%; function coverage is 92.85%; and branch coverage is 89.40% across 32 instrumented files.

Both `adminEventController.js` and `adminUserController.js` have achieved 100% coverage across all statements, branches, and functions, resolving the major gap identified in previous coverage rounds.

Remaining gaps are concentrated in transaction rollback and backfill paths in `registrationIntegrity.js`, edge-case payload validation in `errorHandler.js`, and logging fallback branches in `auditLogger.js`.

Frontend coverage remains focused on `registrationState.js` and is not representative of complete React application UI coverage.

## 6. Recommended improvements

1. Exercise registration transaction failure, replica-set disconnects, and counter recovery/backfill paths in `registrationIntegrity.js`.
2. Add audit logger tests to trigger defensive error catch blocks and incomplete metadata fallbacks in `auditLogger.js`.
3. Add error-handler integration tests covering unhandled exceptions, malformed request payloads, and non-standard HTTP errors in `errorHandler.js`.
4. Extend frontend registration-state tests to cover uncovered boundary and error branches; consider UI integration tests for components outside this measured module.

## 7. Conclusion

The backend reached 90.86% line coverage and 92.85% function coverage in the measured run, surpassing target baselines. The frontend registration state source module reached 76.56% line and 100% function coverage. With both Admin controllers fully verified at 100%, remaining test development can focus on registration integrity and error resiliency.